

Università degli Studi di Milano Bicocca
Scuola di Scienze
Dipartimento di Informatica, Sistemistica e Comunicazione

Metodi del Calcolo Scientifico

Progetto 1

Solutori iterativi per sistemi lineari simmetrici e definiti positivi

Relazione

Matias Bonoli

Anno Accademico 2025-2026

26 giugno 2026

Indice

1	Introduzione	2
1.1	Linguaggio e dipendenze	2
2	I metodi iterativi	2
2.1	I quattro metodi e il criterio di arresto	2
2.2	Metodi stazionari	3
2.3	Metodi del gradiente	3
2.4	Un solo prodotto matrice–vettore per iterazione	3
3	La libreria	3
3.1	L’architettura della libreria	3
3.2	Note implementative rilevanti	4
3.3	Interfaccia da riga di comando	4
4	Esperimenti	5
4.1	Le matrici di test	5
4.2	Risultati numerici	5
4.3	Lettura delle tabelle	8
4.4	Grafici fondamentali	8
4.4.1	Numero di iterazioni	9
4.4.2	Errore relativo	10
4.4.3	Tempo di calcolo	11
4.5	Grafici di approfondimento	13
4.5.1	Storia di convergenza	13
4.5.2	Iterazioni e numero di condizionamento	14
4.5.3	Amplificazione errore/residuo	15
4.5.4	Sintesi di tempo e iterazioni	16
4.5.5	Costo per iterazione	17
5	Comparativa generale	18
A	Codice completo	19
A.1	solvers.py	19
A.2	run_assignment.py	23
A.3	plots.py	26
A.4	analyze_cond.py	33
A.5	test_assignment.py	34

1 Introduzione

L'obiettivo del progetto è sviluppare una libreria che implementa quattro metodi iterativi richiesti dalla traccia, Jacobi, Gauss–Seidel, Gradiente e Gradiente coniugato, per risolvere sistemi lineari $A\mathbf{x} = \mathbf{b}$ con A simmetrica e definita positiva (SPD). La libreria viene poi usata per analizzare il comportamento di questi metodi su un insieme di matrici reali al variare della tolleranza richiesta.

La relazione segue lo stesso percorso. Prima si inquadrano i quattro metodi sul piano algoritmico, a partire dalla derivazione teorica e dal criterio di arresto comune a tutti. Poi si descrive l'architettura della libreria, costruita in modo che lo scheletro dell'iterazione sia scritto una sola volta e ogni metodo fornisca solo la propria regola di aggiornamento, insieme all'eseguibile da riga di comando che misura iterazioni, errori e tempi su un insieme di matrici reali. Infine si raccolgono e analizzano i risultati sperimentali.

1.1 Linguaggio e dipendenze

Il progetto è scritto interamente in Python 3.12. I pacchetti utilizzati sono tutti open source.

- `numpy` per la gestione di array e le operazioni vettoriali (prodotto scalare, norma euclidea, aggiornamenti elemento per elemento).
- `scipy.sparse` per la struttura dati della matrice in formato sparso CSR e il prodotto matrice–vettore $A\mathbf{v}$.
- `scipy.io.mmread` per la lettura dei file in formato MatrixMarket `.mtx`.
- `numba` per la compilazione JIT della sweep di Gauss–Seidel, intrinsecamente sequenziale, così da renderla veloce restando codice nostro.
- `matplotlib` per la generazione dei grafici di sintesi.
- `scipy.sparse.linalg.eigsh` usato solo nello script di analisi a margine per stimare gli autovalori estremi e quindi $\text{cond}(A)$; non fa parte dei solutori.

Come consentito dalla consegna, `scipy/numpy` vengono usati esclusivamente per le strutture dati e le operazioni elementari. Gli algoritmi risolutivi sono interamente sviluppati nel progetto. Non viene usato alcun solutore di sistemi lineari di libreria (niente `spsolve`, `cg`, `numpy.linalg.solve` e simili).

2 I metodi iterativi

2.1 I quattro metodi e il criterio di arresto

Tutti i metodi implementati assumono A simmetrica e definita positiva e partono dallo stesso vettore iniziale nullo $\mathbf{x}^{(0)} = \mathbf{0}$. A ogni iterazione si calcola il residuo

$$\mathbf{r}^{(k)} = \mathbf{b} - A\mathbf{x}^{(k)},$$

cioè la differenza tra il termine noto e il risultato di A applicato alla stima corrente. Se la stima fosse esatta il residuo sarebbe zero; altrimenti indica quanto manca ancora.

Il criterio di arresto è comune a tutti i metodi: l'iterazione si ferma quando il residuo scalato

$$\frac{\|\mathbf{r}^{(k)}\|}{\|\mathbf{b}\|} < \text{tol}.$$

Dividere per $\|\mathbf{b}\|$ rende il confronto indipendente dalla scala del problema. A protezione dei casi di non convergenza, l'iterazione si arresta comunque dopo `max_iter`= 20000 passi, segnalando in tal caso `converged`= False.

2.2 Metodi stazionari

L'idea dei metodi stazionari è semplice: si spezza $A = P - N$ scegliendo P facile da invertire. A ogni passo si prende la stima attuale e la si corregge con il residuo filtrato da P^{-1} :

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + P^{-1}\mathbf{r}^{(k)}.$$

Più P approssima bene A , meno iterazioni servono; più è complicata da invertire, più ogni passo è costoso. Il metodo converge se il raggio spettrale della matrice di iterazione $\rho(P^{-1}N) < 1$: più è piccolo, più l'errore si riduce in fretta.

- Jacobi sceglie $P = D$ (solo la diagonale di A). L'inversa è banale: basta prendere i reciproci degli elementi diagonali. Questo rende il passo completamente parallelizzabile, perché tutte le componenti si aggiornano insieme. Per A SPD la convergenza non è garantita in generale; servono condizioni aggiuntive come la dominanza diagonale.
- Gauss-Seidel sceglie $P = D + L$ (la parte triangolare inferiore di A). La differenza chiave è che ogni componente aggiornata viene usata subito per calcolare le successive, invece di aspettare il passo successivo. Questo accelera la convergenza di circa il doppio rispetto a Jacobi. Inoltre, per A SPD, Gauss-Seidel converge sempre. Il prezzo è che le componenti vanno aggiornate in sequenza e non si può parallelizzare.

2.3 Metodi del gradiente

Per A SPD, risolvere $A\mathbf{x} = \mathbf{b}$ equivale a trovare il minimo del funzionale quadratico

$$\varphi(\mathbf{y}) = \frac{1}{2} \mathbf{y}^\top A \mathbf{y} - \mathbf{b}^\top \mathbf{y},$$

il cui gradiente è $\nabla\varphi(\mathbf{y}) = A\mathbf{y} - \mathbf{b}$, che si annulla esattamente nella soluzione. Geometricamente, φ è una superficie a forma di ciotola: la soluzione esatta è il fondo. Le curve di livello sono ellissi concentriche la cui eccentricità dipende dal condizionamento: più la matrice è mal condizionata, più la ciotola è allungata e stretta.

- Il Gradiente (steepest descent) a ogni passo scende nella direzione più ripida, che coincide con il residuo $\mathbf{r}^{(k)}$. Il passo ottimo lungo quella direzione è

$$\alpha_k = \frac{\mathbf{r}^{(k)} \cdot \mathbf{r}^{(k)}}{\mathbf{r}^{(k)} \cdot A\mathbf{r}^{(k)}}, \quad \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_k \mathbf{r}^{(k)}.$$

Converge sempre per A SPD, ma sulle matrici mal condizionate la direzione più ripida non punta verso il fondo della valle ma di traverso: si genera il caratteristico zig-zag e la convergenza rallenta proporzionalmente al condizionamento $\text{cond}(A) = \lambda_{\max}/\lambda_{\min}$.

- Il Gradiente coniugato (CG) sceglie ogni nuova direzione di ricerca in modo che non interferisca con quelle già usate ($\mathbf{d}_i^\top A \mathbf{d}_j = 0$ per $i \neq j$). Una volta trovato il minimo in una direzione, i passi successivi non lo disfano: niente zig-zag. Per A SPD di dimensione n la soluzione esatta viene raggiunta in al più n passi, e in pratica ne bastano molte meno. Il numero di iterazioni scala come $\sqrt{\text{cond}(A)}$, molto meglio del Gradiente semplice.

2.4 Un solo prodotto matrice-vettore per iterazione

Moltiplicare A per un vettore è l'operazione più costosa di ogni iterazione, con costo proporzionale al numero di elementi non nulli. Per Gradiente e Gradiente coniugato questa moltiplicazione si fa già per calcolare il passo ottimo α_k . Invece di ricalcolare il residuo come $\mathbf{b} - A\mathbf{x}^{(k+1)}$ (un secondo prodotto), lo si aggiorna per ricorrenza riusando il prodotto $A\mathbf{v}$ già disponibile:

$$\mathbf{r}^{(k+1)} = \mathbf{r}^{(k)} - \alpha_k A\mathbf{v}.$$

Su matrici dense questo dimezza il costo dominante per ogni iterazione.

3 La libreria

3.1 L'architettura della libreria

La libreria (file `solvers.py`) non è una sequenza di funzioni indipendenti, ma una piccola gerarchia di classi costruita su un'osservazione: tutti i metodi condividono lo stesso scheletro e differiscono soltanto nella regola con cui aggiornano la soluzione a ogni passo. L'architettura si articola in tre componenti.

1. Lo scheletro comune. La classe base astratta `IterativeSolver` implementa una sola volta il metodo `solve()`: inizializza $\mathbf{x}^{(0)} = \mathbf{0}$, esegue il ciclo che si arresta sul residuo scalato (o al superamento di `max_iter`), misura il tempo e costruisce il risultato.

```
x^(0) = 0
finche' ||b - A x^(k)|| / ||b|| >= tol e k < max_iter:
    x^(k+1) = aggiornamento(x^(k))
    k += 1
```

2. Il passo specifico. L'interfaccia astratta `_Iteration` incapsula lo stato e il passo di un metodo (pattern template method). Le quattro classi concrete `Jacobi`, `GaussSeidel`, `Gradient`, `ConjugateGradient` forniscono unicamente la propria regola di aggiornamento. Aggiungere un quinto metodo significa scrivere una sola classe, senza toccare lo scheletro.
3. Il risultato uniforme. La `dataclass` `SolverResult` raccoglie in modo omogeneo l'esito di ogni risoluzione: soluzione approssimata, iterazioni, tempo, residuo relativo finale e flag di convergenza.

Le istanze dei quattro metodi sono raccolte nella tupla `SOLVERS`, su cui iterano sia il runner sia i test. Le parti comuni sono fattorizzate una sola volta (`_validate_for_iteration` per verificare che la matrice sia quadrata con diagonale non nulla; `_safe_norm` per il calcolo della norma di $\mathbf{b}$).

3.2 Note implementative rilevanti

L'implementazione degli algoritmi si basa su quattro accorgimenti.

- Sparsità preservata. Nessun metodo fattorizza o modifica A : la matrice viene usata solo per prodotti $A\mathbf{v}$, il cui costo è proporzionale al numero di non-zeri e non a n^2 (nessun fill-in). È questo a rendere i metodi iterativi competitivi sui grandi sistemi sparsi.
- Un solo matvec per iterazione. Come descritto in precedenza, per Gradiente e Gradiente coniugato il residuo successivo si ottiene per ricorrenza, evitando un prodotto matrice-vettore aggiuntivo.
- Gauss-Seidel senza inversa, ma veloce. Il passo di aggiornamento scorre le componenti della soluzione una per una, riusando subito i valori appena calcolati. Ogni componente dipende da quelle precedenti, quindi non si può parallelizzare. Essendo un ciclo sequenziale, viene compilato con `numba` (`_gauss_seidel_sweep`), restando codice nostro: in questo modo non diventa il collo di bottiglia che sarebbe un ciclo Python puro.
- Tempi equi. Prima delle misure viene eseguito un warm-up che forza la compilazione JIT su un problemino 3×3 , così il tempo riportato per Gauss-Seidel non include il costo (una tantum) di compilazione.

3.3 Interfaccia da riga di comando

Per applicare e confrontare i metodi è stato realizzato un eseguibile (`run_assignment.py`). L'utente può lanciare la validazione completa su tutte le matrici e tolleranze, oppure restringere il calcolo a una matrice specifica (`--matrix`), a una tolleranza (`--tol`) o a un metodo (`--method`), parametri tutti ripetibili. Il software produce un CSV numerico con tutte le metriche e i grafici di sintesi, salvabili in una cartella di output configurabile.

4 Esperimenti

Per ogni matrice è stata seguita la procedura di validazione richiesta dalla traccia. Si fissa come soluzione esatta $\mathbf{x} = [1, 1, \dots, 1]$ e si ricava il termine noto come $\mathbf{b} = A\mathbf{x}$. I quattro metodi partono da $\mathbf{x}^{(0)} = \mathbf{0}$. Conoscere la soluzione vera permette di misurare l'errore reale

$$\frac{\|\mathbf{x}_{\text{calc}} - \mathbf{x}\|}{\|\mathbf{x}\|},$$

cioè quanto la soluzione calcolata si discosta da quella esatta in proporzione alla sua grandezza. Questo sarebbe impossibile in un problema applicativo in cui la soluzione non è nota. Ogni configurazione è stata ripetuta alle quattro tolleranze $\text{tol} \in \{10^{-4}, 10^{-6}, 10^{-8}, 10^{-10}\}$, con $\text{max_iter} = 20000$.

Le misure di tempo sono state condotte sullo stesso ambiente: Apple M4 (10 core), 16 GB di RAM, macOS 26.5, Python 3.12 con numpy 2.4, scipy 1.17 e numba 0.65. I numeri di iterazione e gli errori sono invece deterministici e indipendenti dalla macchina.

4.1 Le matrici di test

Le quattro matrici fornite si dividono in due famiglie con caratteristiche opposte, come riassunto nella Tabella 1.

Tabella 1: Caratteristiche delle matrici di test (autovalori e $\text{cond}(A)$ stimati con `analyze_cond.py`).

Matrice	n	non-zeri	$\lambda_{\min}$	$\lambda_{\max}$	$\text{cond}(A)$
spa1	1000	182 434	$4.88 \cdot 10^{-1}$	$9.99 \cdot 10^2$	$2.05 \cdot 10^3$
spa2	3000	1 633 298	$2.12 \cdot 10^0$	$3.00 \cdot 10^3$	$1.41 \cdot 10^3$
vem1	1681	13 385	$1.23 \cdot 10^{-2}$	$4.00 \cdot 10^0$	$3.25 \cdot 10^2$
vem2	2601	21 225	$7.89 \cdot 10^{-3}$	$4.00 \cdot 10^0$	$5.07 \cdot 10^2$

Le **spa** sono molto più dense (circa 180 e 540 non-zeri per riga) e con condizionamento maggiore; le **vem** sono molto più sparse (circa 8 non-zeri per riga) e meglio condizionate. Questa differenza strutturale si riflette in modo netto sulle prestazioni dei diversi metodi.

4.2 Risultati numerici

Le tabelle seguenti riportano i risultati completi estratti da `results/results.csv`. Tutti i metodi sono arrivati a convergenza entro $\text{max_iter} = 20000$. I tempi sono in secondi sull'ambiente indicato e sono soggetti a variazioni di runtime fra esecuzioni.

Tabella 2: **spa1** — $n = 1000$, $\text{nnz} = 182\,434$.

tol	metodo	iter	err. rel.	res. scal.	t [s]
10^{-4}	Jacobi	115	$1.77 \cdot 10^{-3}$	$9.48 \cdot 10^{-5}$	0.013
10^{-4}	Gauss–Seidel	9	$1.82 \cdot 10^{-2}$	$7.79 \cdot 10^{-5}$	0.002
10^{-4}	Gradiente	143	$3.46 \cdot 10^{-2}$	$9.91 \cdot 10^{-5}$	0.016
10^{-4}	Gradiente coniugato	49	$2.08 \cdot 10^{-2}$	$9.78 \cdot 10^{-5}$	0.006
10^{-6}	Jacobi	181	$1.80 \cdot 10^{-5}$	$9.62 \cdot 10^{-7}$	0.021
10^{-6}	Gauss–Seidel	17	$1.30 \cdot 10^{-4}$	$5.58 \cdot 10^{-7}$	0.004
10^{-6}	Gradiente	3577	$9.68 \cdot 10^{-4}$	$9.98 \cdot 10^{-7}$	0.407
10^{-6}	Gradiente coniugato	134	$2.55 \cdot 10^{-5}$	$9.74 \cdot 10^{-7}$	0.015
10^{-8}	Jacobi	247	$1.82 \cdot 10^{-7}$	$9.76 \cdot 10^{-9}$	0.028
10^{-8}	Gauss–Seidel	24	$1.71 \cdot 10^{-6}$	$7.34 \cdot 10^{-9}$	0.006
10^{-8}	Gradiente	8233	$9.82 \cdot 10^{-6}$	$9.99 \cdot 10^{-9}$	0.930
10^{-8}	Gradiente coniugato	177	$1.32 \cdot 10^{-7}$	$9.03 \cdot 10^{-9}$	0.020
10^{-10}	Jacobi	313	$1.85 \cdot 10^{-9}$	$9.91 \cdot 10^{-11}$	0.035
10^{-10}	Gauss–Seidel	31	$2.25 \cdot 10^{-8}$	$9.65 \cdot 10^{-11}$	0.008
10^{-10}	Gradiente	12919	$9.82 \cdot 10^{-8}$	$9.99 \cdot 10^{-11}$	1.463
10^{-10}	Gradiente coniugato	200	$1.20 \cdot 10^{-9}$	$7.94 \cdot 10^{-11}$	0.023

Tabella 3: **spa2** — $n = 3000$, $\text{nnz} = 1\,633\,298$.

tol	metodo	iter	err. rel.	res. scal.	t [s]
10^{-4}	Jacobi	36	$1.77 \cdot 10^{-3}$	$9.54 \cdot 10^{-5}$	0.043
10^{-4}	Gauss–Seidel	5	$2.60 \cdot 10^{-3}$	$4.14 \cdot 10^{-5}$	0.013
10^{-4}	Gradiente	161	$1.81 \cdot 10^{-2}$	$9.90 \cdot 10^{-5}$	0.186
10^{-4}	Gradiente coniugato	42	$9.82 \cdot 10^{-3}$	$9.91 \cdot 10^{-5}$	0.049
10^{-6}	Jacobi	57	$1.67 \cdot 10^{-5}$	$9.00 \cdot 10^{-7}$	0.067
10^{-6}	Gauss–Seidel	8	$5.14 \cdot 10^{-5}$	$7.85 \cdot 10^{-7}$	0.020
10^{-6}	Gradiente	1949	$6.69 \cdot 10^{-4}$	$9.99 \cdot 10^{-7}$	2.280
10^{-6}	Gradiente coniugato	122	$1.20 \cdot 10^{-4}$	$9.09 \cdot 10^{-7}$	0.144
10^{-8}	Jacobi	78	$1.57 \cdot 10^{-7}$	$8.49 \cdot 10^{-9}$	0.097
10^{-8}	Gauss–Seidel	12	$2.79 \cdot 10^{-7}$	$4.27 \cdot 10^{-9}$	0.031
10^{-8}	Gradiente	5087	$6.87 \cdot 10^{-6}$	$9.98 \cdot 10^{-9}$	6.028
10^{-8}	Gradiente coniugato	196	$5.59 \cdot 10^{-7}$	$9.60 \cdot 10^{-9}$	0.232
10^{-10}	Jacobi	99	$1.48 \cdot 10^{-9}$	$8.01 \cdot 10^{-11}$	0.119
10^{-10}	Gauss–Seidel	15	$5.57 \cdot 10^{-9}$	$8.52 \cdot 10^{-11}$	0.037
10^{-10}	Gradiente	8285	$6.94 \cdot 10^{-8}$	$9.99 \cdot 10^{-11}$	9.891
10^{-10}	Gradiente coniugato	240	$5.32 \cdot 10^{-9}$	$9.82 \cdot 10^{-11}$	0.291

Tabella 4: **vem1** — $n = 1681$, $\text{nnz} = 13\,385$.

tol	metodo	iter	err. rel.	res. scal.	t [s]
10^{-4}	Jacobi	1314	$3.54 \cdot 10^{-3}$	$9.99 \cdot 10^{-5}$	0.015
10^{-4}	Gauss–Seidel	659	$3.51 \cdot 10^{-3}$	$9.93 \cdot 10^{-5}$	0.018
10^{-4}	Gradiente	890	$2.70 \cdot 10^{-3}$	$9.93 \cdot 10^{-5}$	0.013
10^{-4}	Gradiente coniugato	38	$4.08 \cdot 10^{-5}$	$7.11 \cdot 10^{-5}$	0.001
10^{-6}	Jacobi	2433	$3.54 \cdot 10^{-5}$	$9.99 \cdot 10^{-7}$	0.029
10^{-6}	Gauss–Seidel	1218	$3.53 \cdot 10^{-5}$	$9.99 \cdot 10^{-7}$	0.032
10^{-6}	Gradiente	1612	$2.71 \cdot 10^{-5}$	$9.97 \cdot 10^{-7}$	0.023
10^{-6}	Gradiente coniugato	45	$3.73 \cdot 10^{-7}$	$8.90 \cdot 10^{-7}$	0.001
10^{-8}	Jacobi	3552	$3.54 \cdot 10^{-7}$	$9.99 \cdot 10^{-9}$	0.040
10^{-8}	Gauss–Seidel	1778	$3.52 \cdot 10^{-7}$	$9.96 \cdot 10^{-9}$	0.048
10^{-8}	Gradiente	2336	$2.70 \cdot 10^{-7}$	$9.90 \cdot 10^{-9}$	0.033
10^{-8}	Gradiente coniugato	53	$2.83 \cdot 10^{-9}$	$7.80 \cdot 10^{-9}$	0.001
10^{-10}	Jacobi	4671	$3.54 \cdot 10^{-9}$	$9.99 \cdot 10^{-11}$	0.052
10^{-10}	Gauss–Seidel	2338	$3.51 \cdot 10^{-9}$	$9.94 \cdot 10^{-11}$	0.063
10^{-10}	Gradiente	3058	$2.71 \cdot 10^{-9}$	$9.97 \cdot 10^{-11}$	0.043
10^{-10}	Gradiente coniugato	59	$2.19 \cdot 10^{-11}$	$6.91 \cdot 10^{-11}$	0.001

Tabella 5: **vem2** — $n = 2601$, $\text{nnz} = 21\,225$.

tol	metodo	iter	err. rel.	res. scal.	t [s]
10^{-4}	Jacobi	1927	$4.97 \cdot 10^{-3}$	$9.99 \cdot 10^{-5}$	0.029
10^{-4}	Gauss–Seidel	965	$4.95 \cdot 10^{-3}$	$9.98 \cdot 10^{-5}$	0.036
10^{-4}	Gradiente	1308	$3.81 \cdot 10^{-3}$	$9.98 \cdot 10^{-5}$	0.024
10^{-4}	Gradiente coniugato	47	$5.73 \cdot 10^{-5}$	$9.19 \cdot 10^{-5}$	0.001
10^{-6}	Jacobi	3676	$4.97 \cdot 10^{-5}$	$9.99 \cdot 10^{-7}$	0.055
10^{-6}	Gauss–Seidel	1840	$4.94 \cdot 10^{-5}$	$9.96 \cdot 10^{-7}$	0.069
10^{-6}	Gradiente	2438	$3.79 \cdot 10^{-5}$	$9.92 \cdot 10^{-7}$	0.045
10^{-6}	Gradiente coniugato	56	$4.74 \cdot 10^{-7}$	$8.54 \cdot 10^{-7}$	0.001
10^{-8}	Jacobi	5425	$4.97 \cdot 10^{-7}$	$9.99 \cdot 10^{-9}$	0.082
10^{-8}	Gauss–Seidel	2714	$4.96 \cdot 10^{-7}$	$9.99 \cdot 10^{-9}$	0.104
10^{-8}	Gradiente	3566	$3.81 \cdot 10^{-7}$	$9.97 \cdot 10^{-9}$	0.065
10^{-8}	Gradiente coniugato	66	$4.30 \cdot 10^{-9}$	$8.60 \cdot 10^{-9}$	0.001
10^{-10}	Jacobi	7174	$4.96 \cdot 10^{-9}$	$9.98 \cdot 10^{-11}$	0.110
10^{-10}	Gauss–Seidel	3589	$4.95 \cdot 10^{-9}$	$9.97 \cdot 10^{-11}$	0.134
10^{-10}	Gradiente	4696	$3.80 \cdot 10^{-9}$	$9.94 \cdot 10^{-11}$	0.084
10^{-10}	Gradiente coniugato	74	$2.25 \cdot 10^{-11}$	$5.90 \cdot 10^{-11}$	0.001

4.3 Lettura delle tabelle

Prima di passare ai grafici conviene fissare alcune osservazioni trasversali alle tabelle precedenti. Tutti i metodi raggiungono la convergenza entro `max_iter` = 20000 in ogni configurazione: la rete di sicurezza non viene mai attivata. Il residuo scalato finale è in ogni riga immediatamente sotto la tolleranza richiesta, a conferma che il criterio di arresto funziona come previsto. Il dato più interessante riguarda l'errore relativo, che a parità di `tol` non è uguale fra i metodi. Su `spa1` a `tol` = 10^{-6} il Gradiente si ferma a $9.68 \cdot 10^{-4}$, circa mille volte il residuo imposto, mentre sulle `vem` il Gradiente coniugato scende fino a $\sim 10^{-11}$, persino sotto il residuo. Le tre metriche principali, iterazioni, errore e tempo, meritano però una lettura grafica dedicata, che viene condotta nel seguito un set per volta.

4.4 Grafici fondamentali

I tre grafici fondamentali (script `plots.py`) riportano, per ciascuna matrice, l'andamento di una metrica al ridursi della tolleranza, con l'asse delle ascisse in scala logaritmica orientato da 10^{-4} a 10^{-10} .

4.4.1 Numero di iterazioni

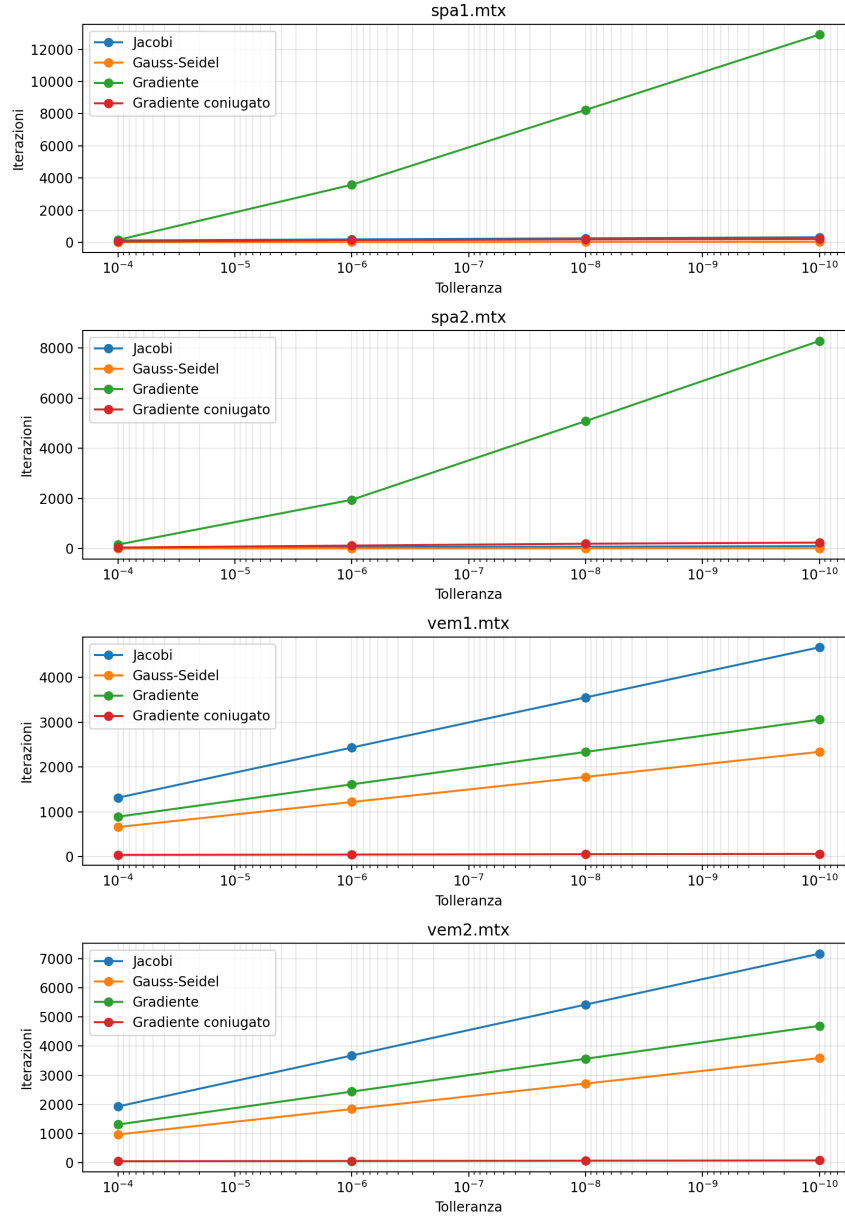


Figura 1: Numero di iterazioni al variare di tol (asse x in scala logaritmica, una riga per matrice; asse y lineare).

Ci si aspetta che il numero di iterazioni dei metodi stazionari dipenda da quanto la diagonale domina il resto della matrice (governato da $\rho(P^{-1}N)$), mentre quello dei metodi del gradiente dipenda dal numero di condizionamento $\text{cond}(A)$. I risultati in Figura 1 confermano queste premesse e mettono in luce due scenari opposti. Sulle **spa** la curva del Gradiente sale in modo marcato, crescendo quasi linearmente fino a 12 919 iterazioni su **spa1** e 8 285 su **spa2** a $\text{tol} = 10^{-10}$, mentre Jacobi, Gauss-Seidel e Gradiente coniugato restano schiacciati in basso, a poche centinaia di passi o meno. Sulle **vem** lo scenario si capovolge: la curva più alta diventa quella di Jacobi (fino a 7 174 su **vem2**), seguita dal Gradiente e poi da Gauss-Seidel, che ne dimezza con regolarità i passi, mentre il Gradiente coniugato si appiattisce su una linea quasi orizzontale di poche decine di iterazioni. Le **spa** sono fortemente dominanti diagonali (stazionari veloci) ma mal condizionate (Gradiente lento per lo zig-zag); le **vem**, meno dominanti ma meglio condizionate,

riequilibrano la gara. In entrambi i regimi il Gradiente coniugato, che scala come $\sqrt{\text{cond}(A)}$, resta nettamente il più parsimonioso.

4.4.2 Errore relativo

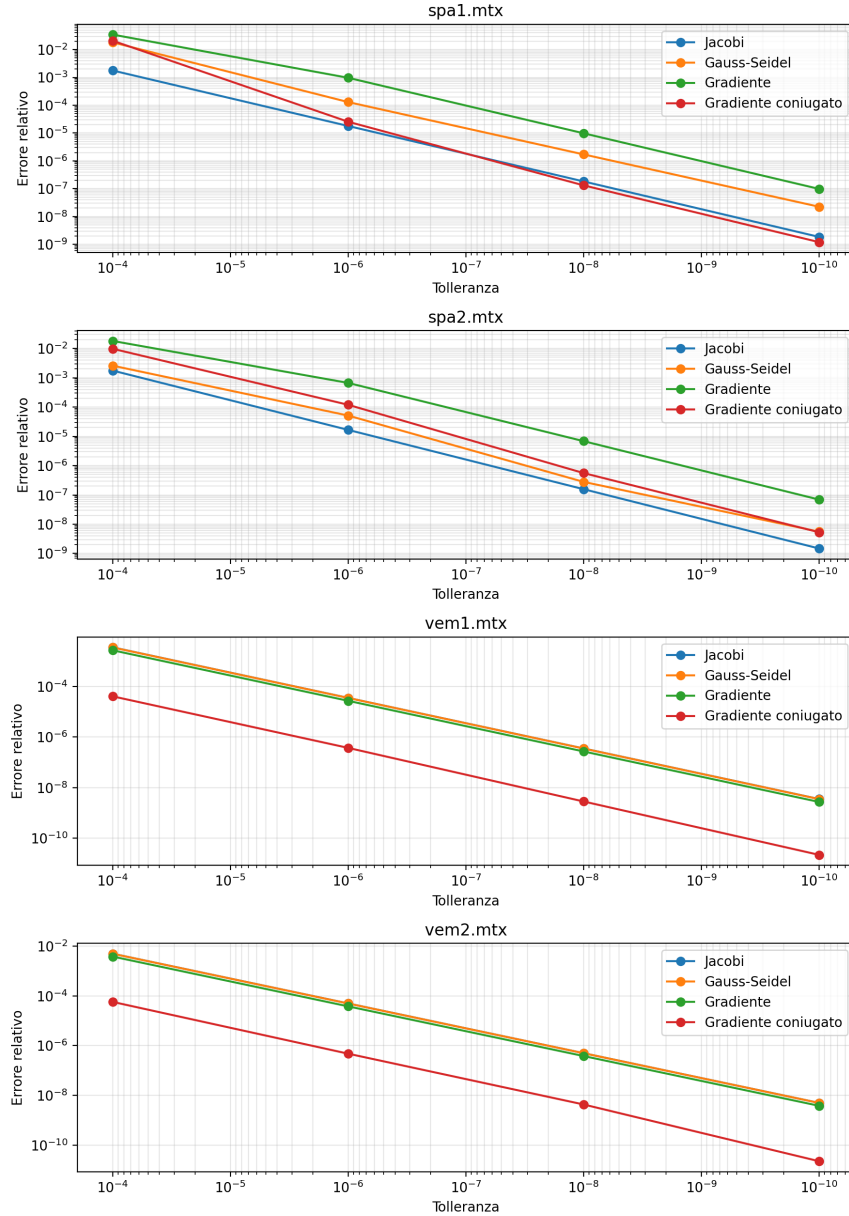


Figura 2: Errore relativo $\|\mathbf{x}_{\text{calc}} - \mathbf{x}\|/\|\mathbf{x}\|$ al variare di `tol` (entrambi gli assi in scala logaritmica, una riga per matrice).

Il criterio di arresto controlla il residuo, non l'errore vero. Il legame tra i due dipende dal condizionamento della matrice: su matrici mal condizionate un residuo piccolo può corrispondere a un errore ancora grande. A parità di tolleranza l'errore non sarà quindi lo stesso fra i metodi. La Figura 2 riporta l'errore relativo vero della soluzione calcolata.

Tutte le curve scendono in modo sostanzialmente lineare al ridursi della tolleranza, ma la loro posizione verticale racconta il ruolo del condizionamento. Sulle `spa` il Gradiente mantiene la curva di errore più alta, mentre Jacobi, Gauss-Seidel e Gradiente coniugato si collocano più in basso, entro circa un ordine di grandezza l'una dall'altra. Sulle `vem` le curve di Jacobi, Gauss-

Seidel e Gradiente si sovrappongono quasi perfettamente, a conferma che condividono lo stesso fattore di amplificazione, mentre quella del Gradiente coniugato corre nettamente al di sotto, 1–2 ordini di grandezza più in basso. A $\text{tol} = 10^{-10}$ su `vem1`, dove gli altri tre si fermano a $\sim 3.5 \cdot 10^{-9}$, il Gradiente coniugato raggiunge $2.19 \cdot 10^{-11}$, un errore inferiore al residuo imposto. Il Gradiente coniugato tende ad abbattere simultaneamente tutte le componenti spettrali dell'errore: a parità di residuo restituisce la soluzione più accurata.

4.4.3 Tempo di calcolo

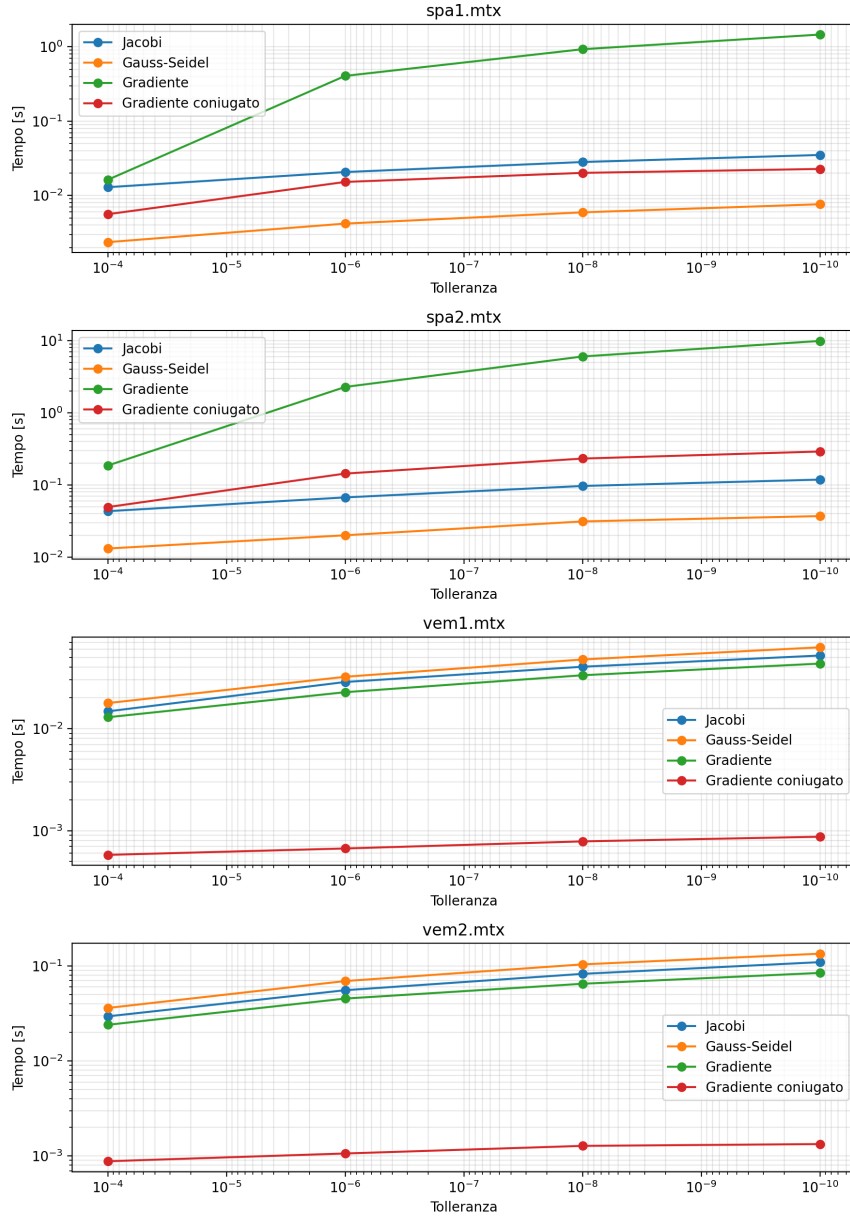


Figura 3: Tempo di calcolo al variare di `tol` (asse y in scala logaritmica, una riga per matrice).

Il tempo totale è il prodotto di due fattori: il numero di iterazioni e il costo del singolo passo. Il costo del passo, dominato dal prodotto matrice–vettore, cresce con il numero di non-zeri della matrice. Le due famiglie hanno densità molto diverse, quindi ci si aspetta uno scenario meno uniforme dei precedenti. I tempi misurati sono in Figura 3.

Sulle **spa**, dense e con matvec costoso, il prezzo delle molte iterazioni del Gradiente diventa severo: la sua curva sale isolata in cima, fino a 9.89 s su **spa2** a $\text{tol} = 10^{-10}$, contro i 0.037 s di Gauss–Seidel e i 0.291 s del Gradiente coniugato. Sulle **vem**, molto sparse e con passo economico, le tre curve di Jacobi, Gauss–Seidel e Gradiente si addensano attorno a 0.05–0.13 s, mentre il Gradiente coniugato corre per distacco al di sotto, nell’ordine di 10^{-3} s. Dove il costo per iterazione è alto vince chi fa meno iterazioni utili; dove è basso il tempo segue da vicino il numero di iterazioni, e il Gradiente coniugato domina senza rivali.

4.5 Grafici di approfondimento

Ai tre grafici fondamentali se ne affiancano cinque ulteriori (script `plots.py`), ciascuno costruito per isolare un aspetto che la lettura metrica vs tolleranza lascia in ombra: la dinamica iterazione per iterazione, la dipendenza esplicita dal condizionamento, l'amplificazione dell'errore e la scomposizione del tempo nei suoi due fattori.

4.5.1 Storia di convergenza

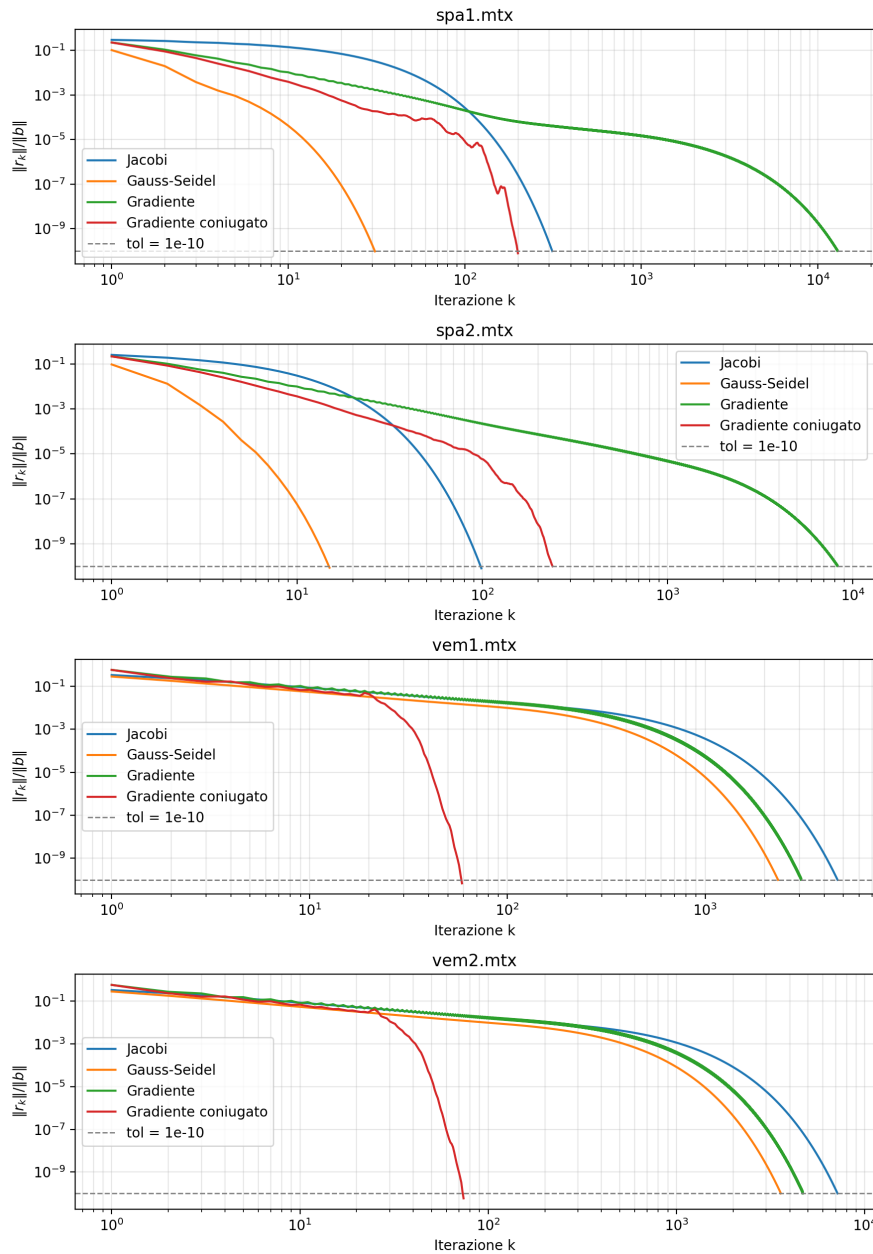


Figura 4: Storia del residuo scalato $\|r_k\|/\|b\|$ in funzione dell'iterazione k ($\text{tol} = 10^{-10}$, entrambi gli assi in scala logaritmica, una riga per matrice; la linea tratteggiata segna la tolleranza).

Ogni famiglia di metodi lascia una firma riconoscibile nella discesa del residuo: un decadimento lineare (in scala semilogaritmica) per i metodi stazionari, un profilo irregolare e lento per il Gradiente sulle matrici mal condizionate, una caduta rapida per il Gradiente coniugato. La storia del residuo scalato è in Figura 4.

Le curve di Jacobi e Gauss–Seidel scendono come rette, con quella di Gauss–Seidel sistematicamente più ripida. Il Gradiente, sulle *spa*, procede a gradoni e si trascina per migliaia di iterazioni prima di toccare la soglia, mentre il Gradiente coniugato precipita e taglia la linea della tolleranza in poche decine di passi. Le direzioni A -coniugate non dis fanno il lavoro già compiuto: ogni passo abbatte una componente dell’errore senza riaprirne altre. Il Gradiente semplice, nella valle stretta delle matrici mal condizionate, rimbalza invece da una parete all’altra.

4.5.2 Iterazioni e numero di condizionamento

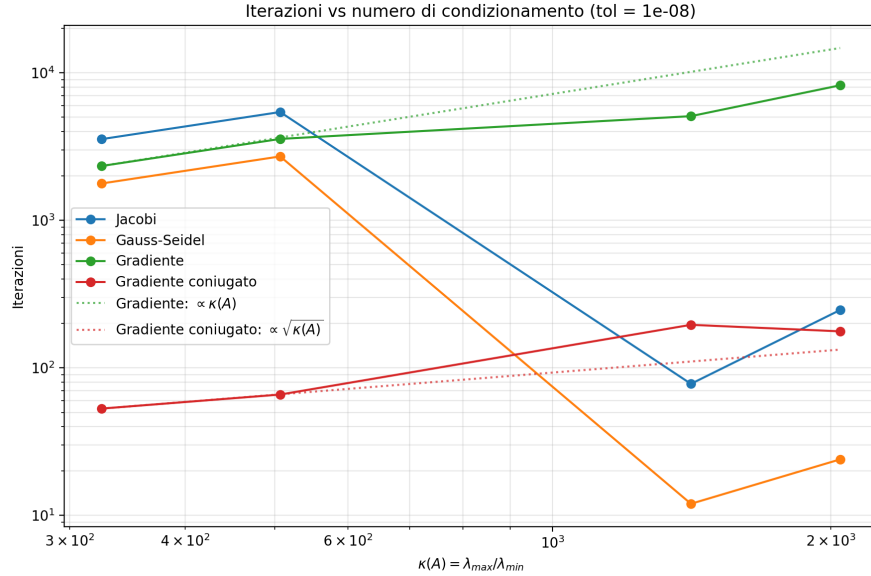


Figura 5: Iterazioni in funzione di $\text{cond}(A)$ ($\text{tol} = 10^{-8}$, assi logaritmici), con le curve teoriche $\propto \kappa(A)$ (Gradiente) e $\propto \sqrt{\kappa(A)}$ (Gradiente coniugato) ancorate al dato a condizionamento minimo.

Il numero di iterazioni del Gradiente cresce proporzionalmente al condizionamento ($\propto \kappa(A)$), mentre quello del Gradiente coniugato cresce solo con la sua radice quadrata ($\propto \sqrt{\kappa(A)}$). I metodi stazionari non seguono il condizionamento ma la struttura diagonale della matrice. La Figura 5 riporta le iterazioni in funzione di $\text{cond}(A)$, affiancando le due curve teoriche di riferimento.

I punti del Gradiente seguono la curva ripida $\propto \kappa(A)$, quelli del Gradiente coniugato quella più dolce $\propto \sqrt{\kappa(A)}$. Al crescere del condizionamento il Gradiente coniugato paga molto meno: raddoppiare $\text{cond}(A)$ raddoppia le iterazioni del Gradiente, ma aumenta solo la radice quadrata per il Gradiente coniugato. Jacobi e Gauss–Seidel non seguono alcuna tendenza rispetto a $\text{cond}(A)$: su di loro conta quanto la diagonale domina il resto della matrice.

4.5.3 Amplificazione errore/residuo

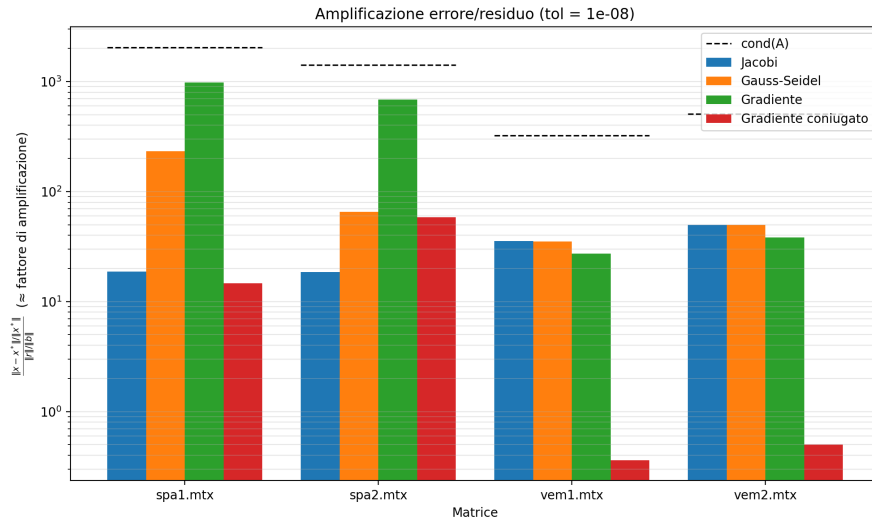


Figura 6: Fattore di amplificazione errore/residuo per metodo e matrice ($\text{tol} = 10^{-8}$, asse y logaritmico); la linea tratteggiata è $\text{cond}(A)$, limite superiore teorico.

Dalla relazione $\text{errore} \lesssim \text{cond}(A) \cdot \text{residuo}$, il condizionamento è il fattore massimo con cui il residuo può essere amplificato nell'errore reale. La Figura 6 riporta questo rapporto di amplificazione con la linea tratteggiata che segna $\text{cond}(A)$ di ciascuna matrice.

In ogni gruppo le barre restano sotto la linea di $\text{cond}(A)$, senza mai violarne il limite. Il fattore di amplificazione cambia però in modo marcato fra i metodi. Su **spa1** il Gradiente amplifica il residuo di circa $9.8 \cdot 10^2$ volte, quasi al soffitto di $\text{cond} \approx 2 \cdot 10^3$, mentre il Gradiente coniugato si ferma a un fattore di circa 15. Da notare il caso delle **vem**, dove la barra del Gradiente coniugato scende sotto l'unità (su **vem1** il rapporto vale circa 0.36): l'errore vero è più piccolo del residuo imposto. A parità di tolleranza il Gradiente coniugato non solo converge prima, ma restituisce anche la soluzione più fedele.

4.5.4 Sintesi di tempo e iterazioni

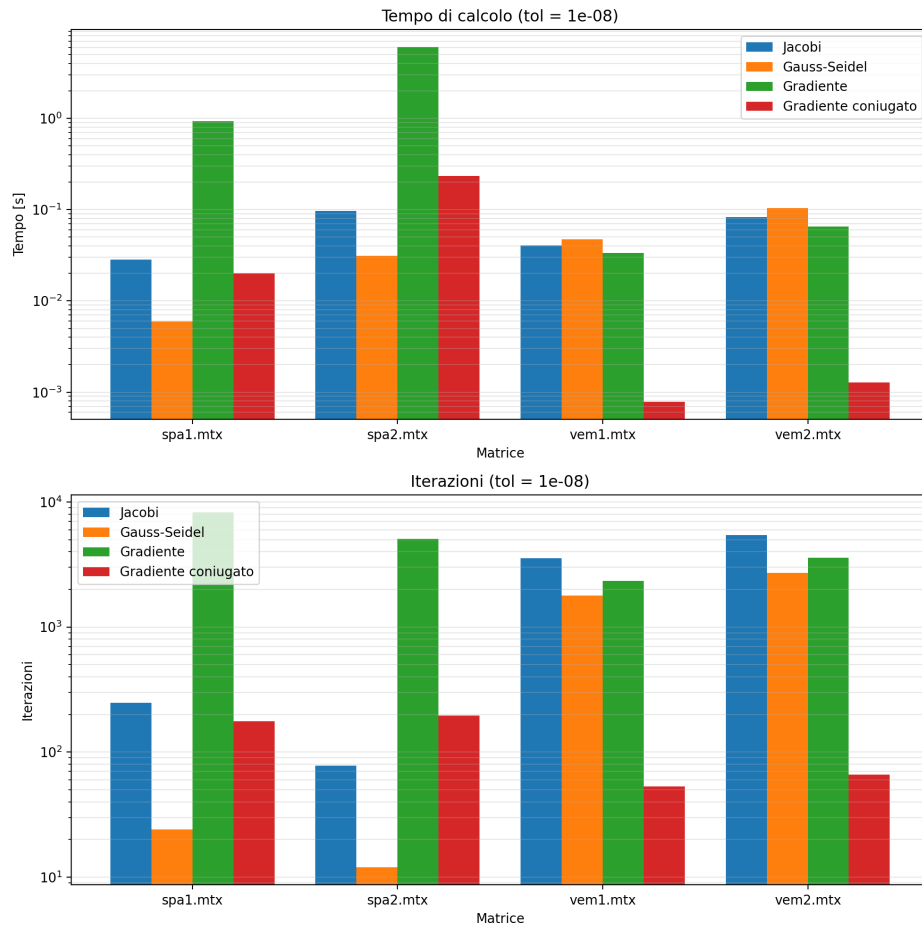


Figura 7: Tempo di calcolo (in alto) e numero di iterazioni (in basso) per metodo e matrice ($\text{tol} = 10^{-8}$, asse y logaritmico).

Fissata una tolleranza rappresentativa, la Figura 7 mette a confronto diretto, a barre, tempo e iterazioni dei quattro metodi. Sulle **spa** la coppia di barre del Gradiente svetta sia in iterazioni sia in tempo, mentre Gauss-Seidel resta la più bassa. Sulle **vem** le barre di Jacobi, Gauss-Seidel e Gradiente si allineano su valori simili e quella del Gradiente coniugato sprofonda, in modo sproporzionato rispetto alle iterazioni. Questo accade perché il matvec sulle **vem** è economico: il tempo segue allora da vicino il numero di passi, premiando il metodo che ne richiede di meno.

4.5.5 Costo per iterazione

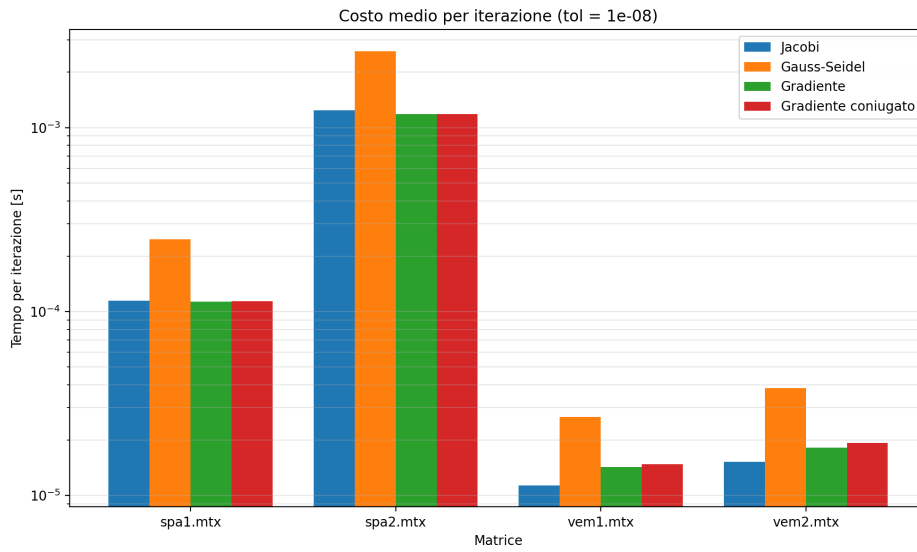


Figura 8: Costo medio per iterazione ($\text{tol} = 10^{-8}$, asse y logaritmico), che isola il peso del singolo passo dal numero di passi.

Per capire se le differenze di tempo osservate dipendano dal numero di iterazioni o dal costo del singolo passo, la Figura 8 riporta il tempo medio per iterazione, cioè il tempo totale diviso per il numero di passi.

A parità di matrice il costo per iterazione è sostanzialmente uniforme fra i metodi, con Gauss-Seidel circa il doppio degli altri per via della sweep sequenziale che si somma al prodotto matrice-vettore. Il costo cresce con la densità della matrice, dall'ordine di 10^{-5} s sulle **vem** fino a $\sim 10^{-3}$ s su **spa2**, in linea con il numero di non-zeri. Poiché il singolo passo costa più o meno lo stesso per tutti, le grandi differenze di tempo totale viste in precedenza non vengono dal costo dell'iterazione ma dal loro numero.

5 Comparativa generale

Mettendo a fattor comune le letture precedenti emerge un quadro coerente.

Due leggi distinte per due famiglie di metodi. La velocità di Jacobi e Gauss–Seidel dipende dalla dominanza diagonale della matrice, quella di Gradiente e Gradiente coniugato dal numero di condizionamento. Per questo le stesse matrici producono classifiche opposte: sulle **spa** (dominanti ma mal condizionate) gli stazionari sono veloci e il Gradiente arranca; sulle **vem** (poco dominanti ma ben condizionate) accade il contrario.

Gauss–Seidel batte sistematicamente Jacobi. Riusando immediatamente le componenti appena calcolate, Gauss–Seidel dimezza con regolarità le iterazioni di Jacobi (su **vem2** a **tol** = 10^{-8} , 2 714 contro 5 425). Il suo passo è però sequenziale e un po’ più caro: sulle **vem** molto sparse i tempi dei due si avvicinano, mentre sulle **spa**, dove converge in pochissimi passi, è nettamente il più rapido tra gli stazionari.

Il Gradiente coniugato è il vincitore in ogni scenario. Converte sempre in poche decine di iterazioni (38–240), con crescita lenta al ridursi di **tol**. Segue la legge $\propto \sqrt{\text{cond}(A)}$ e resta sempre sotto il limite teorico di n passi. A parità di tolleranza restituisce l’errore più piccolo, ed è il più veloce sulle matrici sparse e il secondo sulle dense dopo Gauss–Seidel.

Il Gradiente semplice è il più fragile. La direzione di massima discesa, sulle matrici mal condizionate, innesca lo zig-zag: il numero di iterazioni cresce con $\text{cond}(A)$ e, quando il matvec è costoso, il tempo degrada in modo severo. È competitivo solo sulle matrici ben condizionate.

In sintesi, su questo insieme di prove il Gradiente coniugato è la scelta di riferimento. Tra i metodi stazionari, Gauss–Seidel domina Jacobi. Il Gradiente semplice va riservato ai casi ben condizionati. L’andamento sperimentale, su tutte e tre le metriche, rispetta le predizioni teoriche dei metodi.

A Codice completo

Si riporta di seguito il codice sorgente completo della libreria e degli script di supporto. Il cuore dei solutori è `solvers.py`; `run_assignment.py` è l'eseguibile di validazione; `plots.py` produce i grafici fondamentali e quelli di approfondimento; `analyze_cond.py` e `test_assignment.py` sono gli strumenti a contorno (condizionamento e test).

A.1 solvers.py

```
1 from __future__ import annotations
2
3 from abc import ABC, abstractmethod
4 from dataclasses import dataclass
5 from time import perf_counter
6 from typing import Any
7
8 import numpy as np
9 from numba import njit
10
11
12 # -----
13 # Risultato di una risoluzione.
14 # -----
15 @dataclass
16 class SolverResult:
17     method: str # nome del metodo
18     x: np.ndarray # soluzione approssimata  $x^{(k)}$ 
19     iterations: int # numero di iterazioni eseguite
20     elapsed: float # tempo di calcolo in secondi
21     residual_relative: float #  $\|b - Ax\|/\|b\|$  raggiunto
22     converged: bool # True se il criterio di arresto e' stato soddisfatto
23     history: list[float] | None = None # storia del residuo scalato (solo se richiesta)
24
25
26 # -----
27 # Iterazione astratta + classe base con lo scheletro comune.
28 # -----
29 class _Iteration(ABC):
30     """Stato interno di un metodo iterativo per un singolo problema.
31
32     Mantiene (e aggiorna in place) il vettore soluzione 'x' e quanto serve per
33     calcolare il residuo ed eseguire il passo di aggiornamento.
34     """
35
36     @abstractmethod
37     def residual_norm(self) -> float:
38         """Norma euclidea del residuo corrente  $\|b - Ax^{(k)}\|$ ."""
39
40     @abstractmethod
41     def step(self) -> None:
42         """Aggiorna lo stato:  $x^{(k)} \rightarrow x^{(k+1)}$ ."""
43
44
45 class IterativeSolver(ABC):
46     """Classe base: implementa lo scheletro comune a tutti i metodi."""
47
48     name: str = "iterative"
49     key: str = "iterative" # identificativo per la CLI
50     DEFAULT_MAX_ITER: int = 20000
51
52     @abstractmethod
```

```

53 def _make_iteration(self, A: Any, b: np.ndarray, x: np.ndarray) -> _Iteration:
54     """Costruisce l'oggetto iterazione specifico del metodo."""
55
56 def solve(self, A: Any, b: np.ndarray, tol: float,
57           max_iter: int | None = None,
58           record_history: bool = False) -> SolverResult:
59     """Risolva  $Ax = b$  partendo da  $x^{(0)} = 0$ .
60
61     Si arresta quando il residuo scalato scende sotto 'tol' oppure quando
62     si superano 'max_iter' iterazioni (in tal caso converged=False).
63
64     Se 'record_history=True' salva in 'SolverResult.history' il residuo
65     scalato a ogni iterazione (partendo dal valore iniziale = 1). Il valore
66     e' gia' calcolato dal ciclo, quindi non comporta matvec aggiuntivi; va
67     comunque usato SOLO per i grafici di convergenza, NON per le misure di
68     tempo (l'append alla lista introduce un piccolo overhead di Python).
69     """
70     _validate_for_iteration(A)
71     if max_iter is None:
72         max_iter = self.DEFAULT_MAX_ITER
73
74     b = np.asarray(b, dtype=float)
75     x = np.zeros_like(b, dtype=float) # vettore iniziale nullo
76     b_norm = _safe_norm(b)
77
78     start_time = perf_counter()
79     it = self._make_iteration(A, b, x)
80     iterations = 0
81     residual_relative = it.residual_norm() / b_norm # all'inizio vale 1
82     history = [residual_relative] if record_history else None
83     while residual_relative >= tol and iterations < max_iter:
84         it.step()
85         iterations += 1
86         residual_relative = it.residual_norm() / b_norm
87         if history is not None:
88             history.append(residual_relative)
89     elapsed = perf_counter() - start_time
90
91     return SolverResult(
92         method=self.name,
93         x=x.copy(),
94         iterations=iterations,
95         elapsed=elapsed,
96         residual_relative=residual_relative,
97         converged=residual_relative < tol,
98         history=history,
99     )
100
101
102 # -----
103 # 1) Metodo di Jacobi.  $P = \text{diag}(A)$ ,  $x \leftarrow x + P^{-1} r$ ,  $r = b - Ax$ .
104 # -----
105 class _JacobiIteration(_Iteration):
106     def __init__(self, A, b, x):
107         self.A = A
108         self.b = b
109         self.x = x
110         self.inv_diag = 1.0 / np.asarray(A.diagonal(), dtype=float) #  $P^{-1}$ 
111         self.residual = b - A @ x # residuo iniziale (= b)
112
113     def residual_norm(self):
114         return float(np.linalg.norm(self.residual))
115

```

```

116     def step(self):
117         #  $x^{(k+1)} = x^{(k)} + P^{-1} r^{(k)}$ 
118         self.x += self.inv_diag * self.residual
119         self.residual = self.b - self.A @ self.x
120
121
122     class Jacobi(IterativeSolver):
123         name = "Jacobi"
124         key = "jacobi"
125
126         def _make_iteration(self, A, b, x):
127             return _JacobiIteration(A, b, x)
128
129
130     # -----
131     # 2) Metodo di Gauss-Seidel.  $P$  = parte triangolare inferiore di  $A$ .
132     # sweep in place (= sostituzione in avanti), poi  $r = b - A x$ .
133     # -----
134     class _GaussSeidelIteration(_Iteration):
135         def __init__(self, A, b, x):
136             self.A = A
137             self.b = b
138             self.x = x
139             self.diag = np.asarray(A.diagonal(), dtype=float)
140             self.indptr = A.indptr
141             self.indices = A.indices
142             self.data = np.asarray(A.data, dtype=float)
143             self.residual = b - A @ x
144
145         def residual_norm(self):
146             return float(np.linalg.norm(self.residual))
147
148         def step(self):
149             _gauss_seidel_sweep(self.indptr, self.indices, self.data,
150                               self.diag, self.b, self.x)
151             self.residual = self.b - self.A @ self.x
152
153
154     class GaussSeidel(IterativeSolver):
155         name = "Gauss-Seidel"
156         key = "gauss-seidel"
157
158         def _make_iteration(self, A, b, x):
159             return _GaussSeidelIteration(A, b, x)
160
161
162     # -----
163     # 3) Metodo del Gradiente (steepest descent).
164     #  $\alpha = (r \cdot r) / (r \cdot A r)$  ;  $x \leftarrow x + \alpha r$  ;  $r \leftarrow r - \alpha A r$ .
165     # -----
166     class _GradientIteration(_Iteration):
167         def __init__(self, A, b, x):
168             self.A = A
169             self.b = b
170             self.x = x
171             self.residual = b - A @ x # = -grad(phi) in x
172
173         def residual_norm(self):
174             return float(np.linalg.norm(self.residual))
175
176         def step(self):
177             Ar = self.A @ self.residual
178             denom = float(np.dot(self.residual, Ar))

```

```

179     alpha = float(np.dot(self.residual, self.residual)) / denom
180     self.x += alpha * self.residual
181     #  $r^{(k+1)} = b - A x^{(k+1)} = r^{(k)} - \alpha A r^{(k)}$ : niente matvec extra
182     self.residual = self.residual - alpha * Ar
183
184
185 class Gradient(IterativeSolver):
186     name = "Gradiente"
187     key = "gradient"
188
189     def _make_iteration(self, A, b, x):
190         return _GradientIteration(A, b, x)
191
192
193 # -----
194 # 4) Metodo del Gradiente coniugato.
195 # Direzioni A-coniugate: niente "zig-zag", convergenza in  $\leq n$  passi.
196 # -----
197 class _ConjugateGradientIteration(_Iteration):
198     def __init__(self, A, b, x):
199         self.A = A
200         self.b = b
201         self.x = x
202         self.residual = b - A @ x # residuo iniziale (= b)
203         self.direction = self.residual.copy() #  $d^{(0)} = r^{(0)}$ 
204         self.residual_norm_sq = float(np.dot(self.residual, self.residual))
205
206     def residual_norm(self):
207         return float(np.sqrt(self.residual_norm_sq))
208
209     def step(self):
210         Ad = self.A @ self.direction
211         denom = float(np.dot(self.direction, Ad))
212         alpha = self.residual_norm_sq / denom
213         self.x += alpha * self.direction
214         self.residual = self.residual - alpha * Ad
215         new_norm_sq = float(np.dot(self.residual, self.residual))
216         beta = new_norm_sq / self.residual_norm_sq
217         self.direction = self.residual + beta * self.direction
218         self.residual_norm_sq = new_norm_sq
219
220
221 class ConjugateGradient(IterativeSolver):
222     name = "Gradiente coniugato"
223     key = "conjugate-gradient"
224
225     def _make_iteration(self, A, b, x):
226         return _ConjugateGradientIteration(A, b, x)
227
228
229 # Istanze dei quattro metodi, nell'ordine richiesto dalla consegna.
230 SOLVERS: tuple[IterativeSolver, ...] = (
231     Jacobi(),
232     GaussSeidel(),
233     Gradient(),
234     ConjugateGradient(),
235 )
236
237
238 # -----
239 # Routine compilata (numba) e funzioni di servizio condivise.
240 # -----
241 @njit(cache=True)

```

```

242 def _gauss_seidel_sweep(indptr, indices, data, diag, b, x):
243     """Una iterazione (sweep) di Gauss-Seidel, aggiornando x in place.
244
245     Equivale a risolvere  $(D + L)y = r$  per sostituzione in avanti: ogni  $x[i]$  usa
246     i valori già aggiornati  $x[0..i-1]$  e quelli vecchi  $x[i+1..n-1]$ . E'
247     un'operazione sequenziale ( $x[i]$  dipende dai precedenti) e quindi non
248     vettorizzabile: la compiliamo con numba per renderla veloce restando codice
249     nostro (non usiamo nessun solutore di libreria).
250     """
251     n = x.shape[0]
252     for i in range(n):
253         sigma = 0.0
254         for k in range(indptr[i], indptr[i + 1]):
255             j = indices[k]
256             if j != i:
257                 sigma += data[k] * x[j]
258         x[i] = (b[i] - sigma) / diag[i]
259
260
261 def warmup() -> None:
262     """Forza la compilazione JIT (numba) su un problemino 3x3.
263
264     Va chiamata PRIMA delle misure di tempo: altrimenti la prima esecuzione di
265     Gauss-Seidel paga il costo (una tantum) di compilazione della sweep,
266     falsando il tempo riportato.
267     """
268     from scipy.sparse import csr_matrix
269
270     A = csr_matrix(np.array([[4.0, 1.0, 0.0],
271                             [1.0, 4.0, 1.0],
272                             [0.0, 1.0, 4.0]]))
273     b = A @ np.ones(3)
274     for solver in SOLVERS:
275         solver.solve(A, b, 1e-12, 100)
276
277
278 def _validate_for_iteration(A: Any) -> None:
279     if A.shape[0] != A.shape[1]:
280         raise ValueError("La matrice deve essere quadrata")
281     diag = np.asarray(A.diagonal(), dtype=float)
282     if np.any(diag == 0):
283         raise ValueError("La matrice ha almeno un elemento diagonale nullo")
284
285
286 def _safe_norm(x: np.ndarray) -> float:
287     norm = float(np.linalg.norm(x))
288     if norm == 0:
289         return 1.0
290     return norm

```

A.2 run_assignment.py

```

1  """Command line runner for the first MCS assignment."""
2
3  from __future__ import annotations
4
5  import argparse
6  import csv
7  from pathlib import Path
8
9  import numpy as np

```



```

10 from scipy.io import mmread
11
12 from plots import create_plots
13 from solvers import SOLVERS, warmup
14
15
16 DEFAULT_TOLS = (1e-4, 1e-6, 1e-8, 1e-10)
17 DEFAULT_MAX_ITER = 20000
18 METHOD_NAMES = tuple(solver.key for solver in SOLVERS)
19
20
21 def main() -> None:
22     args = parse_args()
23     base_dir = Path(__file__).resolve().parent
24     output_dir = args.output
25     if not output_dir.is_absolute():
26         output_dir = base_dir / output_dir
27     output_dir.mkdir(parents=True, exist_ok=True)
28
29     matrix_paths = _resolve_matrices(args, base_dir)
30     tolerances = args.tol if args.tol else list(DEFAULT_TOLS)
31     solvers = _select_solvers(args.method)
32
33     warmup() # compilazione JIT (numba) prima delle misure di tempo
34
35     rows: list[dict[str, str]] = []
36     for matrix_path in matrix_paths:
37         print(f"\nMatrice: {matrix_path}")
38         A = mmread(matrix_path).tocsr().astype(float)
39         x_exact = np.ones(A.shape[0], dtype=float)
40         b = A @ x_exact
41
42         for tol in tolerances:
43             print(f" tol = {tol:g}")
44             for solver in solvers:
45                 result = solver.solve(A, b, tol, args.max_iter)
46                 relative_error = np.linalg.norm(x_exact - result.x) / np.linalg.norm(x_exact)
47                 row = {
48                     "matrix": matrix_path.name,
49                     "n": str(A.shape[0]),
50                     "nnz": str(A.nnz),
51                     "tol": f"{tol:.0e}",
52                     "method": result.method,
53                     "relative_error": f"{relative_error:.16e}",
54                     "residual_relative": f"{result.residual_relative:.16e}",
55                     "iterations": str(result.iterations),
56                     "elapsed_seconds": f"{result.elapsed:.6f}",
57                     "converged": str(result.converged),
58                 }
59                 rows.append(row)
60             print(
61                 " "
62                 f"{result.method:20s} "
63                 f"it={result.iterations:6d} "
64                 f"err={relative_error:.3e} "
65                 f"res={result.residual_relative:.3e} "
66                 f"time={result.elapsed:.3f}s "
67                 f"conv={result.converged}"
68             )
69
70     csv_path = output_dir / "results.csv"
71     _write_csv(rows, csv_path)
72     print(f"\nCSV scritto in: {csv_path}")

```

```

73
74     if args.plots:
75         plot_paths = create_plots(rows, output_dir)
76         print(f"Grafici scritti in: {output_dir / 'plots'}")
77         for plot_path in plot_paths:
78             print(f" - {plot_path.name}")
79
80
81 def parse_args() -> argparse.Namespace:
82     parser = argparse.ArgumentParser(description="Mini libreria per sistemi lineari SPD")
83     parser.add_argument(
84         "--matrix",
85         action="append",
86         type=Path,
87         help="Matrice .mtx da usare. Ripetibile. Se assente usa tutti i file .mtx in dati.",
88     )
89     parser.add_argument(
90         "--tol",
91         action="append",
92         type=float,
93         help="Tolleranza da usare. Ripetibile. Se assente usa 1e-4, 1e-6, 1e-8, 1e-10.",
94     )
95     parser.add_argument(
96         "--max-iter",
97         type=int,
98         default=DEFAULT_MAX_ITER,
99         help="Numero massimo di iterazioni, default 20000.",
100    )
101    parser.add_argument(
102        "--method",
103        action="append",
104        choices=METHOD_NAMES,
105        help=(
106            "Metodo da usare. Ripetibile. Se assente usa tutti i metodi. "
107            f"Valori: {' ', ' '.join(METHOD_NAMES)}."
108        ),
109    )
110    parser.add_argument(
111        "--output",
112        type=Path,
113        default=Path("results"),
114        help="Cartella di output, default assignment 1/results.",
115    )
116    parser.add_argument(
117        "--no-plots",
118        dest="plots",
119        action="store_false",
120        help="Non generare grafici.",
121    )
122    parser.set_defaults(plots=True)
123    return parser.parse_args()
124
125
126 def _resolve_matrices(args: argparse.Namespace, base_dir: Path) -> list[Path]:
127     if args.matrix:
128         paths = args.matrix
129     else:
130         data_dir = base_dir / "dati"
131         paths = sorted(data_dir.glob("*.mtx"))
132         if not paths:
133             raise FileNotFoundError(f"Nessuna matrice .mtx trovata in: {data_dir}")
134
135     resolved = []

```

```

136     for path in paths:
137         if not path.is_absolute():
138             path = base_dir / path
139         if not path.exists():
140             raise FileNotFoundError(f"Matrice non trovata: {path}")
141         resolved.append(path)
142     return resolved
143
144
145 def _select_solvers(methods: list[str] | None):
146     if not methods:
147         return SOLVERS
148
149     selected = set(methods)
150     return tuple(solver for solver in SOLVERS if solver.key in selected)
151
152
153 def _write_csv(rows: list[dict[str, str]], csv_path: Path) -> None:
154     fieldnames = [
155         "matrix",
156         "n",
157         "nnz",
158         "tol",
159         "method",
160         "relative_error",
161         "residual_relative",
162         "iterations",
163         "elapsed_seconds",
164         "converged",
165     ]
166     with csv_path.open("w", encoding="utf-8", newline="") as file:
167         writer = csv.DictWriter(file, fieldnames=fieldnames)
168         writer.writeheader()
169         writer.writerows(rows)
170
171
172 if __name__ == "__main__":
173     main()

```

A.3 plots.py

```

1 from __future__ import annotations
2
3 import argparse
4 import csv
5 from collections import defaultdict
6 from pathlib import Path
7
8 import numpy as np
9 from scipy.io import mmread
10 from scipy.sparse import csr_matrix
11
12 from solvers import SOLVERS, warmup
13
14 BASE_DIR = Path(__file__).resolve().parent
15 DATA_DIR = BASE_DIR / "dati"
16
17 # Ordine e colori dei metodi, coerenti in tutti i grafici di approfondimento.
18 METHOD_ORDER = [solver.name for solver in SOLVERS]
19 METHOD_COLORS = {
20     "Jacobi": "tab:blue",

```

```

21     "Gauss-Seidel": "tab:orange",
22     "Gradiente": "tab:green",
23     "Gradiente coniugato": "tab:red",
24 }
25
26
27 # -----
28 # Funzioni di servizio condivise.
29 # -----
30 def _save(fig, path: Path) -> None:
31     """Layout, salvataggio a 200 dpi e chiusura della figura."""
32     import matplotlib.pyplot as plt
33
34     fig.tight_layout()
35     fig.savefig(path, dpi=200)
36     plt.close(fig)
37
38
39 def load_rows(csv_path: Path) -> list[dict[str, str]]:
40     if not csv_path.exists():
41         raise FileNotFoundError(
42             f"Risultati non trovati: {csv_path}. Esegui prima 'python run_assignment.py'."
43         )
44     with csv_path.open(encoding="utf-8", newline="") as file:
45         return list(csv.DictReader(file))
46
47 def load_cond(cond_csv: Path) -> dict[str, float]:
48     if not cond_csv.exists():
49         raise FileNotFoundError(
50             f"File dei condizionamenti non trovato: {cond_csv}. "
51             "Esegui prima 'python analyze_cond.py'."
52         )
53     with cond_csv.open(encoding="utf-8", newline="") as file:
54         return {row["matrix"]: float(row["cond"]) for row in csv.DictReader(file)}
55
56 def _closest_tol(rows: list[dict[str, str]], target: float) -> str:
57     """Etichetta di tolleranza presente nel CSV piu' vicina a 'target'."""
58     tols = sorted({row["tol"] for row in rows}, key=float)
59     return min(tols, key=lambda t: abs(np.log10(float(t)) - np.log10(target)))
60
61
62 def _methods_present(rows: list[dict[str, str]]) -> list[str]:
63     present = {row["method"] for row in rows}
64     return [m for m in METHOD_ORDER if m in present]
65
66
67 def _grouped_bars(ax, groups, series, value_fn) -> None:
68     """Disegna barre raggruppate: un gruppo per 'groups', una barra per
69     'series'. 'value_fn(group, serie)' ritorna l'altezza (o None)."""
70     n_series = max(len(series), 1)
71     width = 0.8 / n_series
72     x = np.arange(len(groups))
73     for j, serie in enumerate(series):
74         offset = (j - (n_series - 1) / 2) * width
75         heights = [value_fn(group, serie) or np.nan for group in groups]
76         ax.bar(x + offset, heights, width=width, label=serie,
77               color=METHOD_COLORS.get(serie))
78
79
80 # -----
81 # Grafici fondamentali: metrica vs tolleranza (richiamati da run_assignment.py).
82 # -----
83 def create_plots(rows: list[dict[str, str]], output_dir: str | Path) -> list[Path]:

```

```

84     output_dir = Path(output_dir)
85     plots_dir = output_dir / "plots"
86     plots_dir.mkdir(parents=True, exist_ok=True)
87
88     paths = [
89         plots_dir / "relative_error.png",
90         plots_dir / "iterations.png",
91         plots_dir / "time.png",
92     ]
93     _plot_metric(rows, "relative_error", "Errore relativo", paths[0], log_y=True)
94     _plot_metric(rows, "iterations", "Iterazioni", paths[1], log_y=False)
95     _plot_metric(rows, "elapsed_seconds", "Tempo [s]", paths[2], log_y=True)
96     return paths
97
98
99 def _plot_metric(
100     rows: list[dict[str, str]],
101     metric: str,
102     ylabel: str,
103     path: Path,
104     log_y: bool,
105 ) -> None:
106     import matplotlib.pyplot as plt
107
108     grouped = defaultdict(list)
109     for row in rows:
110         grouped[(row["matrix"], row["method"])].append(row)
111
112     matrices = sorted({row["matrix"] for row in rows})
113     fig, axes = plt.subplots(len(matrices), 1, figsize=(9, 3.2 * len(matrices)), squeeze=
        False)
114
115     for ax, matrix_name in zip(axes[:, 0], matrices):
116         for (current_matrix, method), values in grouped.items():
117             if current_matrix != matrix_name:
118                 continue
119             values = sorted(values, key=lambda item: float(item["tol"]))
120             xs = [float(item["tol"]) for item in values]
121             ys = [float(item[metric]) for item in values]
122             ax.plot(xs, ys, marker="o", label=method)
123
124             ax.set_title(matrix_name)
125             ax.set_xlabel("Tolleranza")
126             ax.set_ylabel(ylabel)
127             ax.set_xscale("log")
128             ax.invert_xaxis()
129             if log_y:
130                 ax.set_yscale("log")
131             ax.grid(True, which="both", alpha=0.3)
132             ax.legend()
133
134     _save(fig, path)
135
136
137 # -----
138 # A) Storia di convergenza.
139 # -----
140 def compute_histories(tol: float, max_iter: int) -> dict[str, dict[str, list[float]]]:
141     """Riesegue i solutori registrando la storia del residuo scalato.
142
143     Ritorna histories[matrice][metodo] = [r0, r1, ...] (r0 = 1).
144     """
145     warmup()

```

```

146 paths = sorted(DATA_DIR.glob("*.mtx"))
147 histories: dict[str, dict[str, list[float]]] = {}
148 for path in paths:
149     A = csr_matrix(mread(path), dtype=float)
150     # .shape e' tipato Optional negli stub di scipy: a runtime e' sempre una
151     # tupla (n, n), quindi il pedice e' sicuro.
152     x_exact = np.ones(A.shape[0], dtype=float) # type: ignore[reportOptionalSubscript]
153     b = A @ x_exact
154     per_method: dict[str, list[float]] = {}
155     for solver in SOLVERS:
156         result = solver.solve(A, b, tol, max_iter, record_history=True)
157         per_method[solver.name] = result.history or []
158     histories[path.name] = per_method
159     print(f" storia calcolata: {path.name}")
160 return histories
161
162
163 def plot_convergence(histories: dict[str, dict[str, list[float]]], tol: float,
164                     path: Path) -> None:
165     import matplotlib.pyplot as plt
166
167     matrices = sorted(histories)
168     fig, axes = plt.subplots(len(matrices), 1, figsize=(9, 3.2 * len(matrices)),
169                             squeeze=False)
170     for ax, matrix_name in zip(axes[:, 0], matrices):
171         for method in METHOD_ORDER:
172             history = histories[matrix_name].get(method)
173             if not history or len(history) < 2:
174                 continue
175             # Saltiamo il punto iniziale (r0 = 1, k = 0) perche' l'asse x e' in
176             # scala logaritmica: cosi' tutti i metodi sono visibili anche quando
177             # il numero di iterazioni varia di ordini di grandezza (es. CG ~10^2
178             # vs Gradiente ~10^4 sulle matrici spa).
179             tail = history[1:]
180             ax.plot(range(1, len(tail) + 1), tail, label=method,
181                    color=METHOD_COLORS.get(method))
182             ax.axhline(tol, color="gray", linestyle="--", linewidth=1,
183                       label=f"tol = {tol:g}")
184             ax.set_title(matrix_name)
185             ax.set_xlabel("Iterazione k")
186             ax.set_ylabel(r"$\|r_k\| / \|b\|$")
187             ax.set_xscale("log")
188             ax.set_yscale("log")
189             ax.grid(True, which="both", alpha=0.3)
190             ax.legend()
191 _save(fig, path)
192
193
194 # -----
195 # B) Iterazioni vs cond(A).
196 # -----
197 def plot_iter_vs_cond(rows: list[dict[str, str]], cond_map: dict[str, float],
198                      tol_label: str, path: Path) -> None:
199     import matplotlib.pyplot as plt
200
201     selected = [row for row in rows if row["tol"] == tol_label]
202     fig, ax = plt.subplots(figsize=(9, 6))
203
204     for method in _methods_present(selected):
205         points = [(cond_map[row["matrix"]], int(row["iterations"]))
206                  for row in selected
207                  if row["method"] == method and row["matrix"] in cond_map]
208     points.sort()

```

```

209     if not points:
210         continue
211     conds = [p[0] for p in points]
212     iters = [p[1] for p in points]
213     ax.plot(conds, iters, marker="o", label=method,
214             color=METHOD_COLORS.get(method))
215
216     # Curve teoriche di riferimento, ancorate al punto a cond minimo del
217     # Gradiente / CG (numero di iterazioni ~ cond per il Gradiente, ~sqrt(cond)
218     # per il CG).
219     conds_axis = np.array(sorted(cond_map.values()))
220     _add_reference_curve(ax, selected, cond_map, "Gradiente", conds_axis,
221                         lambda c, c0: c / c0, r"$\propto \kappa(A)$")
222     _add_reference_curve(ax, selected, cond_map, "Gradiente coniugato",
223                         conds_axis, lambda c, c0: np.sqrt(c / c0),
224                         r"$\propto \sqrt{\kappa(A)}$")
225
226     ax.set_title(f"Iterazioni vs numero di condizionamento (tol = {tol_label})")
227     ax.set_xlabel(r"$\kappa(A) = \lambda_{\max} / \lambda_{\min}$")
228     ax.set_ylabel("Iterazioni")
229     ax.set_xscale("log")
230     ax.set_yscale("log")
231     ax.grid(True, which="both", alpha=0.3)
232     ax.legend()
233     _save(fig, path)
234
235
236 def _add_reference_curve(ax, selected, cond_map, method, conds_axis,
237                         shape, label) -> None:
238     """Disegna una curva teorica ancorata al dato reale a cond minimo."""
239     points = [(cond_map[row["matrix"]], int(row["iterations"]))
240              for row in selected
241              if row["method"] == method and row["matrix"] in cond_map]
242     if not points:
243         return
244     points.sort()
245     c0, it0 = points[0]
246     ys = [it0 * shape(c, c0) for c in conds_axis]
247     ax.plot(conds_axis, ys, linestyle=":", color=METHOD_COLORS.get(method),
248             alpha=0.7, label=f"{method}: {label}")
249
250
251 # -----
252 # C) Amplificazione errore/residuo.
253 # -----
254 def plot_error_vs_residual(rows: list[dict[str, str]], cond_map: dict[str, float],
255                             tol_label: str, path: Path) -> None:
256     import matplotlib.pyplot as plt
257
258     selected = [row for row in rows if row["tol"] == tol_label]
259     matrices = sorted({row["matrix"] for row in selected})
260     methods = _methods_present(selected)
261
262     ratios = defaultdict(dict) # ratios[matrix][method] = err/res
263     for row in selected:
264         res = float(row["residual_relative"])
265         if res > 0:
266             ratios[row["matrix"]][row["method"]] = float(row["relative_error"]) / res
267
268     fig, ax = plt.subplots(figsize=(10, 6))
269     _grouped_bars(ax, matrices, methods,
270                  lambda mat, met: ratios.get(mat, {}).get(met))
271

```

```

272 # Riferimento: il rapporto err/res e' limitato superiormente da cond(A).
273 for i, matrix_name in enumerate(matrices):
274     if matrix_name in cond_map:
275         ax.hlines(cond_map[matrix_name], i - 0.4, i + 0.4,
276                 color="black", linestyle="--", linewidth=1.2,
277                 label="cond(A)" if i == 0 else "_nolegend_")
278
279 ax.set_title(f"Amplificazione errore/residuo (tol = {tol_label})")
280 ax.set_xlabel("Matrice")
281 ax.set_ylabel(r" $\frac{\|x-x^*\|}{\|x^*\|} \frac{\|r\|}{\|b\|}$  ( $\approx$  fattore di amplificazione)")
282 ax.set_yscale("log")
283 ax.set_xticks(range(len(matrices)))
284 ax.set_xticklabels(matrices)
285 ax.grid(True, axis="y", which="both", alpha=0.3)
286 ax.legend()
287 _save(fig, path)
288
289
290 # -----
291 # D) Costo per iterazione + bar chart riassuntivi.
292 # -----
293 def plot_cost_per_iter(rows: list[dict[str, str]], tol_label: str, path: Path) -> None:
294     import matplotlib.pyplot as plt
295
296     selected = [row for row in rows if row["tol"] == tol_label]
297     matrices = sorted({row["matrix"] for row in selected})
298     methods = _methods_present(selected)
299
300     cost = defaultdict(dict) # cost[matrix][method] = elapsed/iterations
301     for row in selected:
302         iters = int(row["iterations"])
303         if iters > 0:
304             cost[row["matrix"]][row["method"]] = float(row["elapsed_seconds"]) / iters
305
306     fig, ax = plt.subplots(figsize=(10, 6))
307     _grouped_bars(ax, matrices, methods,
308                 lambda mat, met: cost.get(mat, {}).get(met))
309     ax.set_title(f"Costo medio per iterazione (tol = {tol_label})")
310     ax.set_xlabel("Matrice")
311     ax.set_ylabel("Tempo per iterazione [s]")
312     ax.set_yscale("log")
313     ax.set_xticks(range(len(matrices)))
314     ax.set_xticklabels(matrices)
315     ax.grid(True, axis="y", which="both", alpha=0.3)
316     ax.legend()
317     _save(fig, path)
318
319
320 def plot_summary_bars(rows: list[dict[str, str]], tol_label: str, path: Path) -> None:
321     import matplotlib.pyplot as plt
322
323     selected = [row for row in rows if row["tol"] == tol_label]
324     matrices = sorted({row["matrix"] for row in selected})
325     methods = _methods_present(selected)
326
327     time_of = defaultdict(dict)
328     iter_of = defaultdict(dict)
329     for row in selected:
330         time_of[row["matrix"]][row["method"]] = float(row["elapsed_seconds"])
331         iter_of[row["matrix"]][row["method"]] = int(row["iterations"])
332
333     fig, axes = plt.subplots(2, 1, figsize=(10, 10))
334     _grouped_bars(axes[0], matrices, methods,

```



```

335         lambda mat, met: time_of.get(mat, {}).get(met))
336 axes[0].set_title(f"Tempo di calcolo (tol = {tol_label})")
337 axes[0].set_ylabel("Tempo [s]")
338 axes[0].set_yscale("log")
339
340 _grouped_bars(axes[1], matrices, methods,
341               lambda mat, met: iter_of.get(mat, {}).get(met))
342 axes[1].set_title(f"Iterazioni (tol = {tol_label})")
343 axes[1].set_ylabel("Iterazioni")
344 axes[1].set_yscale("log")
345
346 for ax in axes:
347     ax.set_xlabel("Matrice")
348     ax.set_xticks(range(len(matrices)))
349     ax.set_xticklabels(matrices)
350     ax.grid(True, axis="y", which="both", alpha=0.3)
351     ax.legend()
352 _save(fig, path)
353
354
355 # -----
356 # Main: genera i grafici di approfondimento dai CSV (e dalla riesecuzione dei
357 # solutori per la storia di convergenza).
358 # -----
359 def main() -> None:
360     args = parse_args()
361     output_dir = args.output
362     if not output_dir.is_absolute():
363         output_dir = BASE_DIR / output_dir
364     plots_dir = output_dir / "plots"
365     plots_dir.mkdir(parents=True, exist_ok=True)
366
367     rows = load_rows(output_dir / "results.csv")
368     cond_map = load_cond(output_dir / "cond.csv")
369
370     bar_tol = _closest_tol(rows, args.bar_tol)
371
372     print("Genero i grafici dal CSV...")
373     plot_iter_vs_cond(rows, cond_map, bar_tol, plots_dir / "iter_vs_cond.png")
374     plot_error_vs_residual(rows, cond_map, bar_tol, plots_dir / "error_vs_residual.png")
375     plot_cost_per_iter(rows, bar_tol, plots_dir / "cost_per_iter.png")
376     plot_summary_bars(rows, bar_tol, plots_dir / "summary_bars.png")
377
378     print("Calcolo la storia di convergenza (rieseguo i solutori)...")
379     histories = compute_histories(args.conv_tol, args.max_iter)
380     plot_convergence(histories, args.conv_tol, plots_dir / "convergence.png")
381
382     print(f"\nGrafici scritti in: {plots_dir}")
383     for name in ("convergence.png", "iter_vs_cond.png", "error_vs_residual.png",
384                "cost_per_iter.png", "summary_bars.png"):
385         print(f" - {name}")
386
387
388 def parse_args() -> argparse.Namespace:
389     parser = argparse.ArgumentParser(
390         description="Grafici di approfondimento (MCS Assignment 1)")
391     parser.add_argument("--output", type=Path, default=Path("results"),
392                         help="Cartella con results.csv/cond.csv e output plots/, default results.")
393     parser.add_argument("--conv-tol", type=float, default=1e-10,
394                         help="Tolleranza per la storia di convergenza, default 1e-10.")
395     parser.add_argument("--bar-tol", type=float, default=1e-8,
396                         help="Tolleranza per i grafici a barre / cond, default 1e-8.")

```

```

397     parser.add_argument("--max-iter", type=int, default=20000,
398                         help="Numero massimo di iterazioni, default 20000.")
399     return parser.parse_args()
400
401
402 if __name__ == "__main__":
403     main()

```

A.4 analyze_cond.py

```

1  """Stima del numero di condizionamento delle matrici di test.
2
3  NOTA: questo script serve SOLO all'analisi dei risultati nella relazione
4  (spiegare perche' alcuni metodi sono piu' lenti). NON fa parte della mini
5  libreria di solutori: qui usiamo scipy.sparse.linalg.eigsh, che e' un
6  calcolatore di autovalori, per stimare lambda_max e lambda_min e quindi
7  cond(A) = lambda_max / lambda_min.
8
9  Uso:
10     python analyze_cond.py
11 """
12 from __future__ import annotations
13
14 import csv
15 from pathlib import Path
16
17 from scipy.io import mmread
18 from scipy.sparse.linalg import eigsh
19
20 BASE_DIR = Path(__file__).resolve().parent
21 DATA_DIR = BASE_DIR / "dati"
22 OUTPUT_DIR = BASE_DIR / "results"
23
24
25 def main() -> None:
26     paths = sorted(DATA_DIR.glob("*.mtx"))
27     print(f"{'matrice':<12s}{'n':>7s}{'lambda_min':>14s}{'lambda_max':>14s}{'cond(A)':>14s}"
28         )
29     print("-" * 61)
30     rows: list[dict[str, object]] = []
31     for path in paths:
32         A = mmread(path).tocsr().astype(float)
33         n = A.shape[0]
34         lam_max = float(eigsh(A, k=1, which="LM", return_eigenvectors=False)[0])
35         # piu' piccolo autovalore via shift-invert attorno a 0 (A SPD => lambda_min > 0)
36         lam_min = float(eigsh(A, k=1, sigma=0.0, which="LM",
37                               return_eigenvectors=False)[0])
38         cond = lam_max / lam_min
39         print(f"{path.name:<12s}{n:>7d}{lam_min:>14.3e}{lam_max:>14.3e}{cond:>14.3e}")
40         rows.append({
41             "matrix": path.name,
42             "n": n,
43             "lambda_min": f"{lam_min:.16e}",
44             "lambda_max": f"{lam_max:.16e}",
45             "cond": f"{cond:.16e}",
46         })
47
48     # Salviamo i valori anche su CSV: servono ai grafici (plots.py) e ad
49     # arricchire le tabelle della relazione.
50     OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
51     cond_csv = OUTPUT_DIR / "cond.csv"

```

```

51     with cond_csv.open("w", encoding="utf-8", newline="") as file:
52         writer = csv.DictWriter(
53             file, fieldnames=["matrix", "n", "lambda_min", "lambda_max", "cond"])
54         writer.writeheader()
55         writer.writerows(rows)
56     print(f"\nCSV scritto in: {cond_csv}")
57
58
59 if __name__ == "__main__":
60     main()

```

A.5 test_assignment.py

```

1  """Small tests for the assignment code."""
2
3  from __future__ import annotations
4
5  from pathlib import Path
6  from tempfile import TemporaryDirectory
7
8  import numpy as np
9  from scipy.io import mmread
10 from scipy.sparse import diags
11
12 from solvers import SOLVERS, ConjugateGradient
13
14
15 def test_scipy_reads_matrix_market() -> None:
16     text = """%%MatrixMarket matrix coordinate real general
17 % tiny test matrix
18 3 3 5
19 1 1 2.0
20 1 2 -1.0
21 2 1 -1.0
22 2 2 2.0
23 3 3 3.0
24 """
25     with TemporaryDirectory() as tmp:
26         path = Path(tmp) / "tiny.mtx"
27         path.write_text(text, encoding="utf-8")
28         A = mmread(path).tocsr()
29
30     x = np.array([1.0, 2.0, 3.0])
31     assert np.allclose(A @ x, np.array([0.0, 3.0, 9.0]))
32     assert np.allclose(A.diagonal(), np.array([2.0, 2.0, 3.0]))
33
34
35 def test_solvers_on_tridiagonal_spd() -> None:
36     A = _tridiagonal_spd(20)
37     x_exact = np.ones(20)
38     b = A @ x_exact
39
40     for solver in SOLVERS:
41         result = solver.solve(A, b, tol=1e-8, max_iter=20000)
42         relative_error = np.linalg.norm(x_exact - result.x) / np.linalg.norm(x_exact)
43         assert result.converged, solver.name
44         assert relative_error < 1e-6, solver.name
45
46
47 def test_conjugate_gradient_is_fast_on_small_matrix() -> None:
48     A = _tridiagonal_spd(20)

```

```

49     x_exact = np.ones(20)
50     b = A @ x_exact
51     result = ConjugateGradient().solve(A, b, tol=1e-10, max_iter=20000)
52     assert result.converged
53     assert result.iterations <= 20
54
55
56 def _tridiagonal_spd(n: int):
57     return diags(
58         diagonals=[-np.ones(n - 1), 3 * np.ones(n), -np.ones(n - 1)],
59         offsets=[-1, 0, 1],
60         format="csr",
61     )
62
63
64 if __name__ == "__main__":
65     test_scipy_reads_matrix_market()
66     test_solvers_on_tridiagonal_spd()
67     test_conjugate_gradient_is_fast_on_small_matrix()
68     print("Tutti i test sono passati.")

```